

# Practice 3 – Visual Servoing

Robotics and Cyber-Physical Systems  
School of Sciences and Engineering  
Tecnológico de Monterrey

## Introduction

In this practice, you will use the Stretch head cameras to find and follow a colored object. You will begin by driving the robot manually while observing its RGB and depth images. You will then add scanning, color segmentation, visual servoing, depth measurement, and automatic distance control.

Each part starts from the previous one. Keep each completed script so you can compare how the controller develops.

## Before You Begin

Open a terminal in the repository and update your copy:

```
1 git pull
2 uv sync
```

If you have uncommitted work, save or commit it before pulling. All robot movement in this activity must run with collision management enabled, as it is in `teleop_boilerplate.py`.

## Part 1 – Explore with the Head Cameras

### Objective

Add the head RGB and depth feeds to the teleoperation boilerplate, then use them to explore the robot's environment.

### Background

OpenCV (imported as `cv2`) provides tools for displaying and processing images. NumPy provides efficient numerical arrays; camera images are represented as NumPy arrays of pixel values.

### Task

Copy the boilerplate:

```
1 cp examples/teleop_boilerplate.py p3_1_camera.py
```

Add the camera imports:

```
1 import cv2
2
3 from stretch_tools import Cameras, HEAD_CAMERA
```

Create the camera interface after creating the teleoperation objects:

```
1 cameras = Cameras(head_info=HEAD_CAMERA)
```

Inside the loop, read and display both head feeds:

```
1 success, frame, depth_frame = cameras.read_head()
2 if success:
3     depth_display = cv2.normalize(
4         depth_frame, None, 0, 255, cv2.NORM_MINMAX, cv2.CV_8U
5     )
6     cv2.imshow("Head RGB", frame)
7     cv2.imshow("Head Depth", depth_display)
8
9 if cv2.waitKey(1) & 0xFF == ord("q"):
10     break
```

The raw depth image contains values much larger than the usual 8-bit display range. `cv2.normalize()` maps the current frame to values from 0 to 255, making its structure easier to see. Keep `depth_frame` unchanged for distance calculations; use `depth_display` only for visualization.

Finally, close the cameras and windows in the `finally` block:

```
1 cameras.close()
2 cv2.destroyAllWindows()
```

Run the program with `uv run p3_1_camera.py`. Press `h` to switch the keyboard controls to the head, then use `I`, `J`, `K`, and `L` to look around. Drive carefully and observe how objects appear in both images.

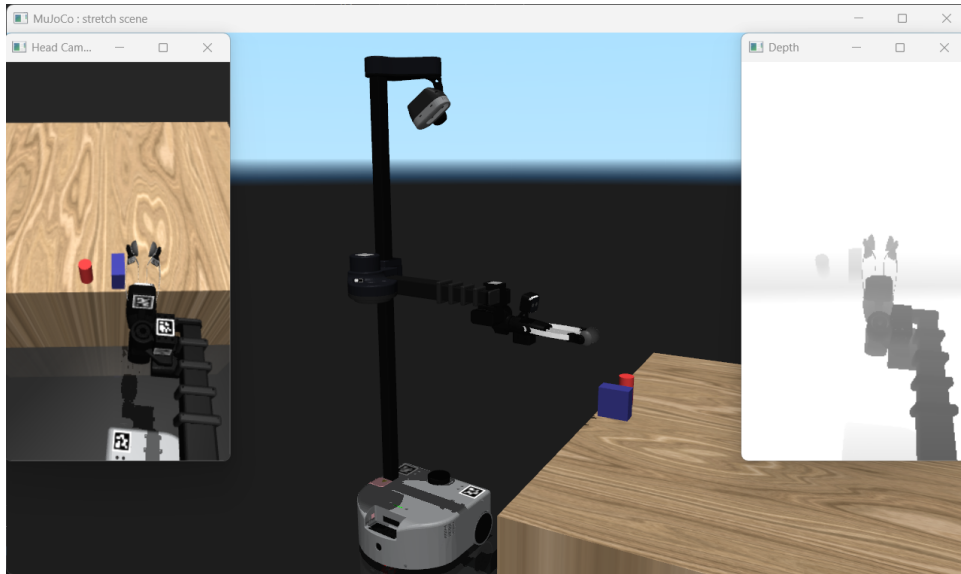


Figure 1: The head RGB and depth camera feeds.

## Part 2 – Scan the Environment

### Objective

Make the robot rotate while keeping its head in a useful scanning pose.

### Background

A `StateController` converts a desired joint position into velocity commands. It continues correcting the joint if it is disturbed, so it is more useful here than issuing a position command only once.

### Task

Copy Part 1 to `p3_2_scan.py`. Import `math` and `StateController`, then create the scanning state:

```
1 scanning_position = StateController(  
2     stretch,  
3     {  
4         "head_pan_counterclockwise": 0.0,  
5         "head_tilt_up": math.radians(-30),  
6     },  
7 )
```

At the beginning of each loop, request the scanning pose and add a slow base rotation:

```
1 command = scanning_position.get_command()  
2 command["base_counterclockwise"] = -0.3  
3 command = teleop.get_manual_override(command)  
4 controller.set_command(command)
```

The robot should rotate continuously while keeping the head facing forward and pitched down. Manual input can still override the autonomous command.

## Part 3 – Detect a Red Object

### Objective

Create an HSV mask, find the largest red region, and print its image position.

### Task

Copy Part 2 to `p3_3_detection.py` and import NumPy:

```
1 import numpy as np
```

Red wraps around the beginning and end of OpenCV's hue range, so it requires two ranges:

```
1 lower_1 = np.array([0, 100, 100])
2 upper_1 = np.array([10, 255, 255])
3 lower_2 = np.array([170, 100, 100])
4 upper_2 = np.array([179, 255, 255])
```

Inside the camera block, create and display the mask:

```
1 hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
2 mask = cv2.inRange(hsv, lower_1, upper_1)
3 mask |= cv2.inRange(hsv, lower_2, upper_2)
4 cv2.imshow("Red Mask", mask)
```

Use `cv2.findContours()`, select the largest contour, and use `cv2.moments()` to calculate its center:

```
1 contours, _ = cv2.findContours(
2     mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
3 )
4 if contours:
5     contour = max(contours, key=cv2.contourArea)
6     moments = cv2.moments(contour)
7     if cv2.contourArea(contour) > 100 and moments["m00"]:
8         x = int(moments["m10"] / moments["m00"])
9         y = int(moments["m01"] / moments["m00"])
10        print(x, y)
```

Move the object through the image and observe how its printed pixel position changes.

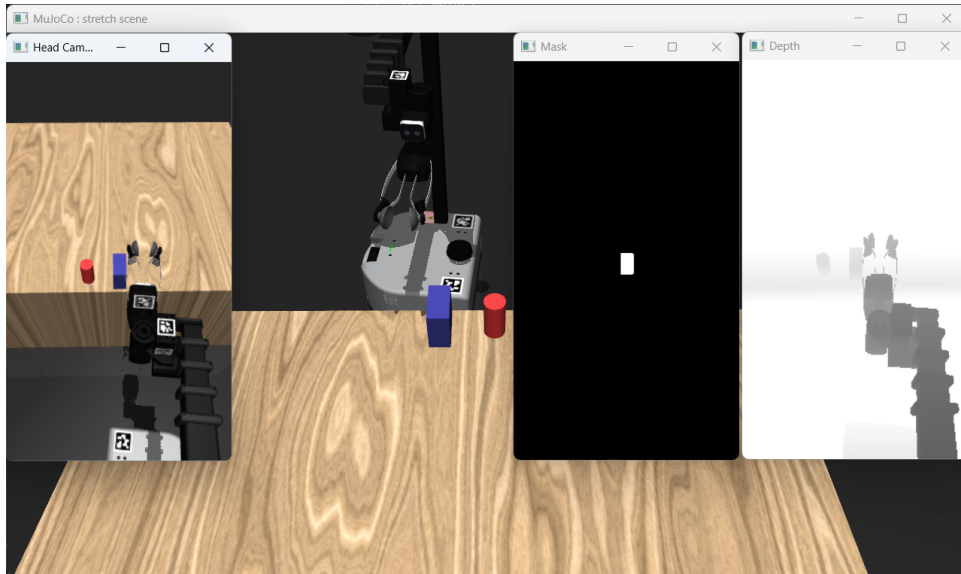


Figure 2: The RGB, depth, and wrapped-red mask windows.

## Part 4 – Follow the Object

### Objective

Refactor object detection into a function, then use proportional control to turn the base and tilt the head toward the object.

### Task

Copy Part 3 to `p3_4_servo.py`. Move the masking and centroid logic into a function:

```
1 def detect_object(frame):
2     # Create the wrapped-red mask and find its largest contour.
3     # Return (x, y) when the contour is valid.
4     # Return None when no object is found.
```

The function should return only the object's center. The mask window is no longer needed. Create a second state controller that holds head pan at zero:

```
1 head_forward = StateController(
2     stretch,
3     {"head_pan_counterclockwise": 0.0},
4 )
```

When the object is visible, normalize its offset from the image center and generate commands:

```
1 KP = 0.5
2
3 x, y = center
4 height, width = frame.shape[:2]
5 horizontal_error = (x - width / 2) / (width / 2)
6 vertical_error = (y - height / 2) / (height / 2)
7
8 command = head_forward.get_command()
9 command["base_counterclockwise"] = -KP * horizontal_error
10 command["head_tilt_up"] = -KP * vertical_error
```

This is proportional control: **the further away the object is from the center, the harder the robot will turn toward it.** As the object approaches the center, the command naturally becomes smaller. The base handles horizontal alignment while the head remains pointed forward; head tilt handles vertical alignment.

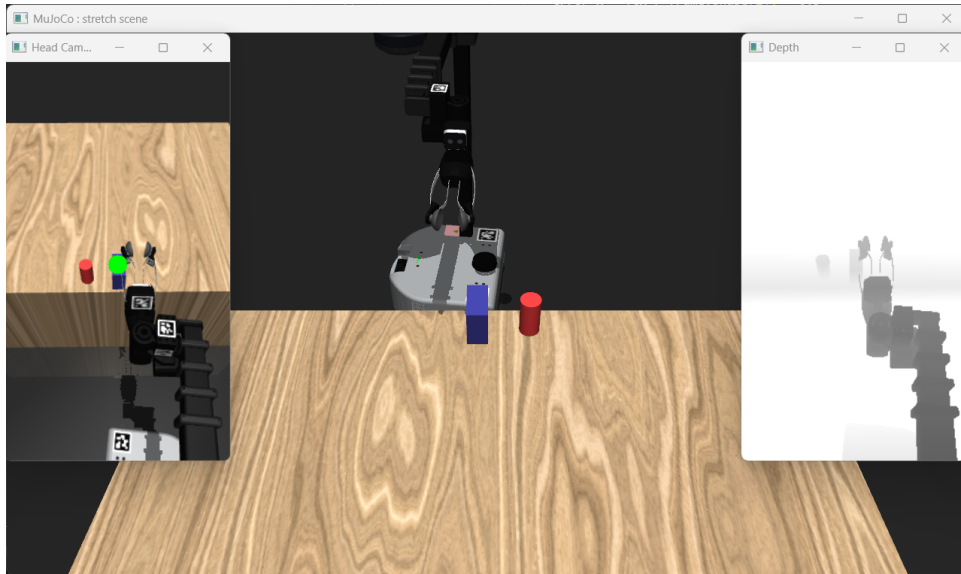


Figure 3: The robot turning to keep the detected object centered.

## Part 5 – Measure Distance

### Objective

Use the head depth image to measure the distance to the detected object.

### Background

The raw depth image stores a distance measurement at each pixel. A value of zero usually means that no valid measurement was available. The normalized image displayed on screen is useful for visualization, but its brightness does not represent a fixed metric scale because each frame is normalized independently. The public `Cameras` interface represents raw depth in millimetres, while `HEAD_CAMERA.get_depth()` converts the measurement to metres and accounts for the different RGB and depth camera intrinsics.

### Task

Copy Part 4 to `p3_5_depth.py`. Read both frames together, then look up the depth at the detected RGB position:

```
1 success, frame, depth_frame = cameras.read_head()
2
3 center = detect_object(frame)
4 if center:
5     distance = HEAD_CAMERA.get_depth(center, depth_frame)
6     if distance is not None:
7         print(f"Distance: {distance:.3f} m")
```

Move the object closer and farther away and compare the physical distance with the printed value.

## Part 6 – Approach the Object

### Objective

Make the robot approach or retreat from the object until it is approximately 0.75 metres away.

### Task

Copy Part 5 to `p3_6_distance_control.py` and add:

```
1 KP_DISTANCE = 10.0
2 TARGET_DISTANCE = 0.75
```

Calculate the distance error and the horizontal alignment factor:

```
1 distance_error = distance - TARGET_DISTANCE
2 alignment = 1.0 - abs(horizontal_error)
3
4 command["base_forward"] = alignment * np.clip(
5     KP_DISTANCE * distance_error, -1.0, 1.0
6 )
```

Set `base_forward` to zero whenever the object or its depth is not available.

The **alignment factor is essential**. When the object is near the edge of the image, the factor approaches zero and the robot prioritizes rotation. As the object becomes centered, the factor approaches one and grants the distance controller more authority to drive forward or backward. Without this factor, the robot could move strongly toward an object before it is actually facing it.

Test the controller from several starting angles and distances. The robot should first face the object, then settle close to the target distance while continuing to track it vertically.

## **Deliverable**

Submit your six Python scripts and a short video demonstrating Part 6. The video should show the robot finding the red object, rotating to face it, approaching or retreating, and settling near 0.75 metres. Briefly explain HSV masking, proportional visual error, depth measurement, and the purpose of the alignment factor.